

Universitatea de Vest din Timișoara

Facultatea de Matematică și Informatică

Analiza comparativă a algoritmilor de sortare

Proiect realizat în limbajul Python

Student: Minciuna David-Florian

Email: david.minciuna06@e-uvt.ro

Grupă: 4

An universitar 2025–2026

Timișoara, România

Cuprins

Abstract	2
1 Introducere	2
2 Obiectivele proiectului	2
3 Definirea problemei	3
4 Analiza complexității	3
5 Modelare și implementare	4
5.1 Mediul experimental	4
5.2 Implementarea algoritmului QuickSort	5
6 Studii de caz și rezultate experimentale	5
6.1 Grafic comparativ	6
7 Avantaje și dezavantaje	7
7.1 Bubble Sort	7
7.2 Selection Sort	7
7.3 Insertion Sort	8
7.4 Quick Sort	8
8 Compararea cu literatura de specialitate	8
9 Concluzii	9
10 Bibliografie	10
11 Anexe	10
11.1 Observații tehnice	10

Abstract

This paper presents a comparative analysis of several classical sorting algorithms implemented in the Python programming language. Both the theoretical and practical performances of Bubble Sort, Selection Sort, Insertion Sort, and Quick Sort are studied and evaluated.

The main objective of the project is to highlight the efficiency differences between these algorithms and to identify the situations in which each sorting method can be used effectively. The paper also analyzes the influence of input data structure on algorithm performance and execution time.

Experimental tests were performed on multiple types of datasets, including random, sorted, reverse sorted, and nearly sorted lists. The obtained results demonstrate that Quick Sort provides the best overall performance in most practical situations, while simpler algorithms become inefficient for large datasets.

Keywords: sorting algorithms, Python, QuickSort, time complexity, experimental analysis

1. Introducere

Sortarea datelor reprezintă una dintre cele mai importante operații din informatică, fiind utilizată în numeroase domenii precum baze de date, motoare de căutare, inteligență artificială și procesarea informațiilor.

Performanța unui algoritm de sortare influențează direct eficiența aplicațiilor software, în special atunci când sunt procesate volume mari de date. Din acest motiv, alegerea algoritmului potrivit este esențială.

În această lucrare sunt analizați și comparați patru algoritmi clasici de sortare implementați în Python: Bubble Sort, Selection Sort, Insertion Sort și Quick Sort.

Importanța studierii algoritmilor de sortare constă în faptul că aceștia reprezintă baza multor operații fundamentale din informatică. Performanța unei aplicații poate depinde direct de eficiența metodei de sortare utilizate, în special în cazul procesării unor volume mari de informații.

2. Obiectivele proiectului

Principalele obiective ale proiectului sunt:

- implementarea algoritmilor de sortare în Python;
- analiza complexității teoretice;

- compararea timpilor de execuție;
- evidențierea avantajelor și dezavantajelor fiecărui algoritm;
- realizarea unor teste experimentale pe diferite tipuri de date.

Aceste obiective au fost stabilite pentru a permite o comparație cât mai clară între algoritmii analizați. Pe lângă rezultatele numerice, proiectul urmărește și înțelegerea modului de funcționare al fiecărui algoritm.

3. Definirea problemei

Problema analizată constă în ordonarea crescătoare a unei liste de numere întregi utilizând diferiți algoritmi de sortare.

Algoritmii analizați sunt:

- Bubble Sort;
- Selection Sort;
- Insertion Sort;
- Quick Sort.

Fiecare algoritm utilizează o abordare diferită pentru rezolvarea problemei de sortare. Unele metode sunt mai simple și mai intuitive, însă mai lente, în timp ce altele utilizează tehnici mai avansate pentru a obține performanțe superioare.

În cadrul proiectului au fost urmărite atât timpul de execuție, cât și comportamentul algoritmilor în funcție de structura datelor de intrare.

4. Analiza complexității

Complexitatea temporală reprezintă un criteriu esențial în evaluarea performanței unui algoritm. Aceasta indică modul în care timpul de execuție crește odată cu dimensiunea datelor de intrare.

Algoritm	Caz mediu	Caz nefavorabil
Bubble Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n^2)$

Algoritmii cu complexitate $O(n^2)$ devin ineficienți pentru liste foarte mari, deoarece timpul de execuție crește rapid odată cu numărul elementelor. În schimb, algoritmii de tip $O(n \log n)$ sunt considerați mult mai eficienți pentru aplicații reale.

Se poate observa faptul că algoritmul Quick Sort oferă cele mai bune performanțe în majoritatea situațiilor practice.

5. Modelare și implementare

Implementarea algoritmilor a fost realizată în limbajul Python datorită simplității sintaxei și a bibliotecilor disponibile pentru testare și analiză. Fiecare algoritm a fost implementat separat pentru a permite compararea corectă a performanțelor.

Pentru realizarea experimentelor au fost generate mai multe tipuri de liste:

- liste aleatoare;
- liste deja sortate;
- liste invers sortate;
- liste aproape sortate.

Utilizarea mai multor tipuri de liste permite observarea comportamentului algoritmilor în situații diferite. De exemplu, unele metode pot funcționa foarte bine pe date aproape sortate, dar pot deveni ineficiente pe date complet aleatoare sau invers sortate.

5.1. Mediul experimental

Experimentele au fost realizate pe următorul sistem:

- Procesor: Intel Core i7-9700F;
- Placă video: NVIDIA RTX 2060;
- Memorie RAM: 16 GB;
- SSD: 1TB;
- Sistem de operare: Windows 11;
- Limbaj de programare: Python.

Menționarea mediului experimental este importantă deoarece timpul de execuție poate varia în funcție de performanța sistemului utilizat. Astfel, rezultatele trebuie interpretate în contextul configurației hardware și software prezentate.

5.2. Implementarea algoritmului QuickSort

Mai jos este prezentată implementarea utilizată pentru algoritmul QuickSort:

```
1 def quickSort(l):
2
3     if len(l) <= 1:
4         return l
5
6     pivot = l[0]
7
8     st = []
9     dr = []
10
11    for i in l[1:]:
12
13        if i <= pivot:
14            st.append(i)
15
16        else:
17            dr.append(i)
18
19    return quickSort(st) + [pivot] + quickSort(dr)
```

Algoritmul utilizează paradigma *divide et impera*, împărțind lista în două subliste pe baza unui element pivot. Elementele mai mici sau egale cu pivotul sunt plasate în partea stângă, iar elementele mai mari sunt plasate în partea dreaptă.

Această strategie permite reducerea dimensiunii problemei la fiecare pas, ceea ce explică eficiența ridicată a algoritmului Quick Sort în cazul mediu.

6. Studii de caz și rezultate experimentale

În urma testelor efectuate au fost obținute următoarele rezultate experimentale. Timpul de execuție este exprimat în secunde.

Tip listă	Bubble	Selection	Insertion	Quick
Aleatoare	0.45	0.40	0.20	0.01
Sortată	0.01	0.40	0.01	0.02
Invers sortată	0.50	0.42	0.35	0.03
Aproape sortată	0.10	0.38	0.02	0.01

Rezultatele obținute evidențiază diferențe importante între algoritmii analizați. Se poate observa faptul că algoritmii simpli, precum Bubble Sort și Selection Sort, au timpi de execuție semnificativ mai mari în comparație cu Quick Sort.

De asemenea, structura listei influențează performanța anumitor algoritmi. Insertion Sort are rezultate foarte bune în cazul listelor aproape sortate, ceea ce îl face util în anumite situații practice.

6.1. Grafic comparativ

Pentru o vizualizare mai clară a diferențelor de performanță, rezultatele experimentale au fost reprezentate grafic.

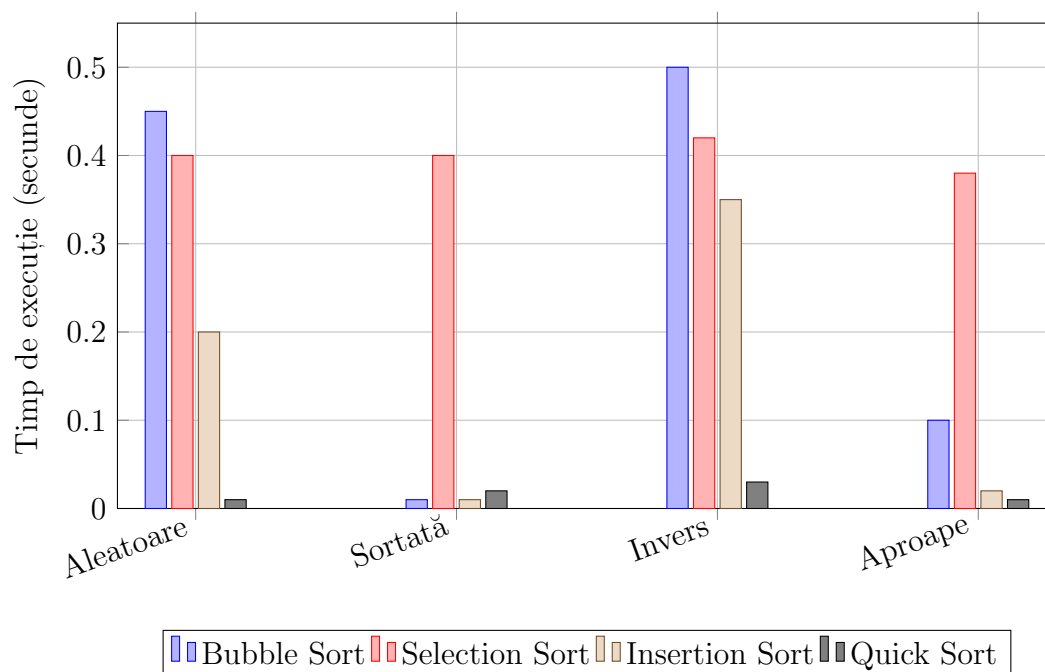


Figura 1: Compararea timpilor de execuție pentru algoritmii analizați

Graficul evidențiază superioritatea algoritmului Quick Sort în majoritatea testelor efectuate. Se observă că timpul său de execuție rămâne foarte redus chiar și în situațiile în care ceilalți algoritmi prezintă creșteri semnificative ale timpului de procesare.

În cazul algoritmului Bubble Sort, performanța scade considerabil pentru liste invers sortate sau aleatoare, ceea ce confirmă complexitatea sa quadratică. Insertion Sort prezintă rezultate bune în cazul listelor aproape sortate, iar Selection Sort menține un comportament relativ constant indiferent de tipul datelor.

Pe baza rezultatelor experimentale se observă următoarele:

- Quick Sort este cel mai eficient în majoritatea situațiilor;
- Insertion Sort funcționează foarte bine pentru liste aproape sortate;
- Bubble Sort devine ineficient pentru volume mari de date;
- Selection Sort are performanțe constante, dar inferioare QuickSort.

7. Avantaje și dezavantaje

7.1. Bubble Sort

Avantaje:

- ușor de implementat;
- intuitiv pentru începători;
- util pentru înțelegerea conceptului de sortare.

Dezavantaje:

- performanță foarte slabă;
- număr mare de comparații;
- nu este recomandat pentru liste mari.

7.2. Selection Sort

Avantaje:

- implementare simplă;
- număr relativ mic de interschimbări;
- ușor de urmărit pas cu pas.

Dezavantaje:

- complexitate $O(n^2)$;
- ineficient pentru seturi mari de date;
- nu profită de faptul că lista poate fi deja parțial sortată.

7.3. Insertion Sort

Avantaje:

- eficient pentru liste mici;
- foarte bun pentru liste aproape sortate;
- implementare simplă.

Dezavantaje:

- devine lent pentru liste mari;
- are complexitate $O(n^2)$ în cazul nefavorabil.

7.4. Quick Sort

Avantaje:

- foarte rapid în practică;
- eficient pentru liste mari;
- utilizează o strategie avansată de tip *divide et impera*.

Dezavantaje:

- implementare mai complexă;
- poate avea caz nefavorabil $O(n^2)$;
- performanța depinde de alegerea pivotului.

8. Compararea cu literatura de specialitate

Algoritmul Quick Sort a fost propus de către *C. A. R. Hoare* în anul 1960 și este considerat unul dintre cei mai eficienți algoritmi de sortare utilizați în practică.

Literatura de specialitate confirmă superioritatea algoritmilor bazați pe paradigma *divide et impera* în comparație cu algoritmii de complexitate quadratică. De asemenea, lucrările consacrate din domeniul algoritmicii prezintă Quick Sort ca fiind o soluție eficientă pentru sortarea datelor în numeroase contexte reale.

Rezultatele obținute experimental sunt în concordanță cu teoria prezentată în lucrările academice consultate. Algoritmii Bubble Sort, Selection Sort și Insertion Sort sunt importanți din punct de vedere educațional, însă Quick Sort este mai potrivit pentru aplicații practice care necesită performanță ridicată.

9. Concluzii

În cadrul acestei lucrări au fost analizați și comparați mai mulți algoritmi clasici de sortare implementați în limbajul Python.

Rezultatele obținute evidențiază diferențele semnificative dintre algoritmi și demonstrează importanța alegerii unei metode eficiente în funcție de dimensiunea și structura datelor.

În urma analizei efectuate se poate concluziona că alegerea algoritmului potrivit depinde atât de dimensiunea datelor, cât și de gradul de ordonare al acestora. Deși algoritmi simpli sunt mai ușor de implementat și înțeleși, algoritmi avansați oferă performanțe semnificativ mai bune în practică.

Ca direcții viitoare de dezvoltare pot fi menționate:

- implementarea MergeSort și HeapSort;
- analiza consumului de memorie;
- testarea pe seturi mari de date;
- compararea rezultatelor cu funcția internă de sortare din Python.

10. Bibliografie

1. C. A. R. Hoare, *Quicksort*, Communications of the ACM, 1961.
2. Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley, 1998.
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press, 2009.

11. Anexe

11.1. Observații tehnice

Pentru a asigura corectitudinea testelor, fiecare algoritm operează pe o copie separată a listei inițiale. În acest mod sunt evitate modificările accidentale ale datelor și sunt obținute rezultate corecte și consistente.

Măsurarea timpului de execuție trebuie realizată în condiții similare pentru fiecare algoritm. Astfel, diferențele observate reflectă în principal eficiența algoritmilor și nu variații cauzate de modificarea datelor de intrare.